

MH6822 Homework 2: OTC Derivatives Compliance Engine

Written Technical and Regulatory Report (approx. 2,500 words)

NGUYEN NGOC TIEN (G2506667B)

KOK WEN SHIN (G2506930B)

May 2026

MH6822 Homework 2: OTC Derivatives Compliance Engine

Technical and Regulatory Report

A modular, multi-jurisdictional reportability checker for Dodd-Frank Title VII (CFTC), EMIR Refit and MAS SF(RDC)R, with first-class handling of the prediction-market classification frontier. The engine ingests a JSON portfolio, parses and classifies each trade, attempts a UPI lookup against an ANNA-DSB-style template library, and produces per-regime compliance verdicts with structured evidence. Run on the 28-trade instructor portfolio plus six team-designed trades (34 trades total), it produces the tallies summarised in Section 3 and isolates the four event contracts that fall outside the current product taxonomy as the substantive regulatory finding rather than as parsing failures.

1. Regulatory Landscape

OTC derivatives reporting rests on three identifier pillars. The Unique Product Identifier (UPI, ISO 4914) names the type of product and is issued solely by ANNA-DSB, designated by the Financial Stability Board in 2019. The Unique Transaction Identifier (UTI, ISO 23897) names a specific trade: a 20-character namespace equal to the generating party's LEI followed by a suffix of at most 32 uppercase alphanumeric characters and hyphens, with both counterparties reporting the same UTI. The Legal Entity Identifier (LEI, ISO 17442) names a legal entity in 20 characters whose final two digits are an ISO 7064 MOD 97-10 check on the preceding eighteen. A reportable trade must carry all three correctly.

The reporting regimes differ in scope and in field requirements. Under the US Commodity Exchange Act, the CFTC requires swap data reporting to a registered swap data repository but does not require trade-level collateral fields, because CFTC collateral reporting is performed at firm level under 17 CFR Part 23. EMIR Refit, by contrast, mandates trade-level collateral fields, the collateral portfolio code and the posted initial and variation margin, even when margin is zero. MAS SF(RDC)R 2013, as set out in MAS Guidelines SFA 06A-G01 of 31 May 2024, follows the EMIR pattern of trade-level collateral reporting for Singapore-booked trades. UPI mandates phased in across jurisdictions between January 2024 (US CFTC) and April 2025 (Singapore and Japan), so a UPI is now an expected field on every conventional OTC trade.

The frontier problem is prediction markets. The ANNA-DSB library covers Rates, Credit, FX, Equity and Commodities but has no definition for event or prediction contracts. The CFTC issued an Advance Notice of Proposed Rulemaking on prediction markets in March 2026 (RIN 3038-AF65, 91 FR 12516); Kalshi, a CFTC-regulated designated contract market, exceeded thirteen billion US dollars notional in March 2026; Polymarket operates offshore on a blockchain and is reachable by EU residents only

through a VPN, which is illegal gambling under most EU frameworks and a separate gambling-law issue in Singapore under the Gambling Control Act 2022. The economic question, as Brandes (2026) frames it, is not the presence of uncertainty but whether the contract performs hedging, price discovery or risk transfer. The engine treats that question as the centre of Module 4 rather than as an edge case.

2. System Architecture

The engine is a pipeline of three stateless functions over three immutable dataclasses. Each stage emits a JSON-serialisable result, so the command-line runner is a simple loop that streams records through the modules and aggregates the output.

```
trades.json --> Module 1 -----> Module 2 -----> Module 3 -----> reports/
                parser           UPI lookup      compliance      JSON, CSV, MD,
                classifier        template lib    multi-regime      Plotly dashboard
```

Module 1, parser and classifier. The parser is permissive: every input record produces a `ParsedTrade`, with data-quality issues accumulated in `parse_errors` and surfaced through a tri-valued `parse_status` of SUCCESS, PARTIAL or FAILED. Malformed inputs, None, lists or scalars, parse without crashing. The classifier emits exactly one of three flags. `CONVENTIONAL_DERIVATIVE` for the whitelisted asset classes, routed to UPI lookup. `NOVEL_INSTRUMENT_NO_TAXONOMY` for explicit prediction-market classes and for unknown classes whose names contain the heuristic tokens event, predict or binary, which routes to the taxonomy-gap branch unless a future `EventContract` template exists. `CLASSIFICATION_AMBIGUOUS` for unknown classes that do not trip the heuristic, treated as a data-quality failure.

Module 2, UPI lookup. The library auto-discovers the product-definitions tree and matches a template by the asset class, instrument type and use-case triple. Templates declare attribute constraints in an ANNA-DSB-style JSON schema with explicit `source_fields` mappings, so one template tolerates upstream field-name variation without code changes. Validation supports type, minimum, maximum, not, minLength, maxLength, pattern, enum and codeset constraints. LIBOR references are treated as warnings rather than errors, because legacy LIBOR-referenced trades remain reportable until maturity under the June 2023 cessation guidance. The output status is FOUND, INVALID_ATTRIBUTES, NOT_FOUND, or NO_PRODUCT_DEFINITION for novel trades. The lookup intentionally checks the template library before the NO_PRODUCT_DEFINITION fallback, so a future active `EventContract` template can override the taxonomy-gap path without a code rewrite.

Module 3, multi-jurisdictional compliance. Regime checkers for CFTC, EMIR Refit and MAS SF(RDC)R share a common CDE field check, UTI namespace match, LEI ISO 7064 validation, ISO 8601 timestamps, currency and amount, action type and cleared flag. EMIR and MAS additionally check the trade-level collateral fields; CFTC does not, which produces the deliberate CFTC-COMPLIANT versus EMIR-NONCOMPLIANT divergence on trades whose collateral fields are null. This three-regimes-share-one collateral pattern is a deliberate, bounded simplification; a production deployment would split the regimes into separate field-set descriptors. For novel trades the CFTC checker reads the structured `platform_type` field: a CFTC-regulated DCM routes to CONDITIONAL with a citation to the active CFTC ANPR, otherwise NOT_APPLICABLE. EMIR and MAS return NOT_APPLICABLE for all novel trades with regime-specific citations. The `ComplianceResult` separates `status` from `reporting_obligation`, so a downstream consumer can answer “is this in scope” and “does it pass” without conflation. The product-definitions tree mirrors the public ANNA-DSB repository layout so the production library can be swapped in with a single clone via the supplied script.

3. Portfolio Findings

The engine was run on the combined 34-trade portfolio (28 instructor trades plus six team-designed trades) under CFTC, EMIR and MAS:

Regime	COMPLIANT	NONCOMPLIANT	CONDITIONAL	NOT_APPLICABLE
CFTC	3	27	3	1
EMIR	2	28	0	4
MAS	2	28	0	4

Of the 34 trades, 30 classify as conventional derivatives and four as novel event contracts. UPI lookup returns FOUND for 25, INVALID_ATTRIBUTES for five and NO_PRODUCT_DEFINITION for the four novel trades. The verdict tally on the original 28-trade instructor fixture is frozen by the regression test `tests/test_engine.py::TestExpectedVerdictCounts` (CFTC: 1 COMPLIANT, 2 CONDITIONAL, 24 NONCOMPLIANT, 1 NOT_APPLICABLE; EMIR: 25 NONCOMPLIANT, 3 NOT_APPLICABLE), which guards against silent verdict drift from later changes.

The six team trades exercise the paths the instructor fixture does not. T029 is a fully clean trade, COMPLIANT under all three regimes, the green path. T030 carries a negative notional, T031 an issuer name where an ISIN is expected, and T032 an off-codeset commodity; all three are caught at the UPI layer and flagged NONCOMPLIANT, demonstrating error types absent from the instructor portfolio. T033 is a Singapore monetary-policy event contract routed to CFTC CONDITIONAL through the structured `platform_type` trigger. T034 is a clean corporate credit swap that completes custom-trade coverage across Rates, FX, Equity, Commodities and Credit, and is COMPLIANT under all three regimes.

The top error fields under EMIR and MAS are the bilateral-margin and identifier fields: variation margin posted (16 trades), collateral portfolio code (15), initial margin posted (15) and the counterparty LEIs (13). These distributions reflect the fixture’s design choice to hit the margin and identifier fields hardest. The four novel trades produce three distinct cross-regime asymmetries an audit team must reconcile: collateral reporting (CFTC firm-level versus EMIR and MAS trade-level), event-contract reportability (CFTC CONDITIONAL via the DCM trigger versus EMIR and MAS NOT_APPLICABLE), and venue access (T027, where no regime expects the data at all). The third is a structural blind spot, not a data-quality problem, and it is the case Module 4 takes up.

4. Module 4 Analysis

This section synthesises the standalone Module 4 written analysis in `docs/module4_classification_analysis.md`; the full per-trade working and references are in that document.

Economic function test. The assignment poses three questions of each event contract: is there an identifiable economic actor with a measurable contractual exposure; would the contract if freely tradeable let that actor manage that exposure; and does it provide price discovery beyond existing sources. Read from the trade records, T026 is a German renewable-energy corporate treasury hedging a subsidy-regime exposure that no conventional derivative addresses; it clears the actor and manageability legs decisively and performs a genuine hedge. T028 is an EU fintech hedging an eight-figure regulatory-classification cost shock; it clears all three legs. T033 is a Singapore corporate hedging a NEER-policy FX exposure economically equivalent to a USD/SGD digital option; it clears the actor and manageability legs. T027 is the exception: an EU asset manager has a real inflation exposure but already holds liquid

continuous hedges, and reaches Polymarket through a circumvented geoblock, so its economic function is mixed and closest to a wager dressed as a hedge. The pattern is that contracts written on regulated DCMs by entities with balance-sheet exposure resemble derivatives in function, while the single contract on an unregulated venue accessed through a bypass is the one that tempts arbitrage.

Schema proposal. The proposal is a minimal ANNA-DSB extension: an EventContract asset class with a BinaryEventContract instrument type, six use-case codes, and a Cash DeliveryType designation, since binary event contracts always settle in cash and physical delivery is out of class. Two attributes are the substance. HedgingExposureType has no analogue in conventional UPI classes because for a swap the hedge-or-speculate distinction is not actionable; for event contracts it determines whether the position is reported to the SDR with derivatives-like granularity or routed to the host jurisdiction’s gambling authority. The MustReferenceObservableSource constraint requires the event to be resolvable from a public, named, third-party source, blocking contracts on unfalsifiable propositions. The machine-readable schema is in `docs/event_contract_upi_template.json`.

Jurisdictional arbitrage and regulatory design. The arbitrage is concrete: in T027 the EU manager escapes both the EMIR perimeter and the venue geoblock while the offshore venue captures unreported flow; the honest hedgers in T026 and T028 are harmed by the same taxonomy gap, holding genuine hedges with no reportable record and no recourse, while regulators lose aggregate exposure visibility. The analysis applies two of Brandes’s five named elements: operator neutrality, contract scope limitations, participant restrictions, market integrity supervision, and position limits and consumer protection. The two selected elements are **participant restrictions** and **market integrity supervision**. Participant restrictions collect the counterparty LEIs, residence and the venue and access fields, validate a residence-versus-venue legality cross-check, carry a participant-eligibility flag in the SDR record, and concretely flag the VPN-bypass access pattern present in T027 as a per-se eligibility failure. Market integrity supervision collects position size and the observable settlement source, validates the observable-source constraint plus a position-limit check, uses large-trader-style periodic SDR reporting, and concretely triggers a mandatory SDR report above one million US dollars per counterparty per contract per quarter. Bringing prediction contracts within EMIR would require the new EventContract UPI class, a UTI rule for pseudonymous-wallet counterparties, and a DLT-native SDR equivalence mechanism.

Engine limits. The absence of a UPI for an asset class that has grown from roughly five listings a year through 2020 to 1,600 in 2025 is itself a systemic-risk indicator, because an unidentifiable asset class cannot be aggregated, position-limited or stress-tested. The recommended response is a phased extension, with a CFTC provisional-UPI and threshold-reporting Stage 1, an ANNA-DSB production-library Stage 2, and an EMIR and MAS perimeter extension Stage 3 paired with explicit memoranda of understanding with the gambling authorities. If advising the CFTC on the ANPR, the two priorities are adding the EventContract UPI class with the DeliveryType and HedgingExposureType designations, and mandating threshold-triggered SDR submission, which together close the measurement gap without waiting for the slower perimeter extensions.

5. Limitations and Future Work

The implementation is a pedagogical fixture, not a production reportability engine, and three areas are deliberately out of scope.

Template distribution. The conventional templates and codesets are hand-rolled stand-ins for the production ANNA-DSB Product Definition distribution. The supplied update script shows the intended swap; in production the attribute names, codeset memberships and UPI codes should come from the

ANNA-DSB monthly distribution rather than from this repository. The proposed EventContract templates are draft schema exemplars for ANNA-DSB review, not registered products.

Identifier registries. The engine validates LEIs against ISO 7064 syntax but does not query GLEIF for issuance status or expiry. UTIs are validated for namespace match but not against an SDR for duplicate detection. ISIN values are checked for shape by regular expression but the check digit is not verified.

Semantic analysis on novel contracts. The engine routes event contracts to `CONDITIONAL` on the structured platform field. It does not parse the natural-language event description to verify that it references an observable, named, third-party source. The `MustReferenceObservableSource` constraint is therefore documented as a manual-review flag rather than an automated check; automating it is its own research problem.

Future work. The architecture is built so the Stage-1 transition in Section 4 requires only template files, not code changes: dropping the four proposed EventContract templates into the product-definitions tree would flip the novel trades from `NO_PRODUCT_DEFINITION` to `FOUND` or `INVALID_ATTRIBUTES` and let the compliance layer reason about them. Beyond that, the natural next steps are per-regime field-set descriptors in place of the shared collateral pattern, a GLEIF issuance-status query, SDR duplicate detection for UTIs, and an assisted classifier for the observable-source constraint.

Team Contribution Statement

Each member contributed to the architecture review and the Module 4 written analysis collectively; the table records primary and secondary ownership.

Member	Matriculation number	Primary contribution	Secondary contribution
NGUYEN NGOC TIEN	G2506667B	Module 1 parser and classifier; Module 2 UPI lookup library; official-trades provenance tests.	CLI verification, product-template fixture review and regression-count checks.
KOK WEN SHIN	G2506930B	Module 3 multi-regime compliance checker; Module 4 classification-frontier analysis; dashboard interpretation.	Regulatory-source hygiene, audit-explanation wording and presentation script.

Running the Engine

Install the only third-party dependency.

```
pip install -r requirements.txt
```

28-trade instructor portfolio, CFTC + EMIR.

```
python run_compliance_check.py --input trades.json --regimes CFTC,EMIR
```

Full 34-trade portfolio under three regimes.

```
python run_compliance_check.py --input portfolio_full.json --regimes CFTC,EMIR,MAS
```

Dashboard from the JSON report.

```
python dashboard.py
```

Test suite.

```
python -m unittest discover -s tests -v
```

Outputs land in `reports/`: `compliance_report.json` (full structured record), `compliance_report.csv` (flat one-row-per-trade summary), `compliance_report.md` (human-readable summary with detailed findings) and `dashboard.html` (Plotly dashboard with the four required figures and a written interpretation).

Source layout

```
compliance_engine/
  src/compliance_engine/      # Python package
    models.py                 # ParsedTrade, UpiLookupResult, ComplianceResult
    utils.py                  # LEI / UTI / ISO 8601 / ISO 4217 validators
    module1_parser.py
    module2_upi_lookup.py
    module3_compliance.py
  data/product_definitions/PROD/OTC-Products/
    UPI/<AssetClass>/*.UPI.V1.json  # conventional templates
    codesets/                      # controlled vocabularies
  tests/test_engine.py          # unit + integration tests
  trades.json                  # instructor file (28)
  additional_trades.json        # team-designed (6)
  portfolio_full.json           # combined (34)
  run_compliance_check.py       # CLI: --regimes CFTC,EMIR
  dashboard.py                  # Plotly dashboard generator
  docs/
    report.md                   # this document
    module4_classification_analysis.md
    event_contract_upi_template.json
    regulatory_sources.md
    marker_guide.md
    presentation_script.md
  reports/
    compliance_report.{json,csv,md}
    dashboard.html
  scripts/update_anna_dsb_product_definitions.sh
```